

Template Specialization and Partial Template Specialization

Template Specialization

In many cases when working with templates, you'll write one generic version for all possible data types and leave it at that--every vector may be implemented in exactly the same way. The idea of template specialization is to override the default template implementation to handle a particular type in a different way.

For instance, while most vectors might be implemented as arrays of the given type, you might decide to save some memory and implement vectors of bools as a vector of integers with each bit corresponding to one entry in the vector. So you might have two separate vector classes. The first class would look like this.

```
template <typename T>
class vector
{
    // accessor functions and so forth
private:
    T* vec_data;    // we'll store the data as block of dynamically allocated
                  // memory
    int length;    // number of elements used
    int vec_size;  // actual size of vec_data
};
```

But when it comes to bools, you might not really want to do this because most systems are going to use 16 or 32 bits for each boolean type even though all that's required is a single bit. So we might make our boolean vector look a little bit different by representing the data as an array of integers whose bits we manually manipulate. (For more on manipulating bits directly, see [bitwise operators and bit manipulations in C and C++](#).)

To do this, we still need to specify that we're working with something akin to a template, but this time the list of template parameters will be empty:

```
template <>
```

and the class name is followed by the specialized type: `class className<type>`. In this case, the template would look like this:

```
template <>
class vector <bool>
{
    // interface

    private:
        unsigned int *vector_data;
        int length;
        int size;
};
```

Note that it would be perfectly reasonable if the specialized version of the vector class had a different interface (set of public methods) than the generic vector class--although they're both vector templates, they don't share any interface or any code.

It's worth pointing out that the salient reason for the specialization in this case was to allow for a more space-efficient implementation, but you could think of other reasons why this might come in handy--for instance, if you wanted to add extra methods to one templated class based on its type, but not to other templates. For instance, you might have a vector of doubles with a method that returns the non-integer component of each element although you might think prefer inheritance in this case. There isn't a particular reason to prevent the existence of a vector of doubles without those extra features. If, however, you felt strongly about the issue and wanted to prevent it, you could do so using template specialization.

Another time when you might want to specialize certain templates could be if you have a template type that relies on some behavior that was not implemented in a collection of classes you'd like to store in that template. For example, if you had a templated sortedVector type that required the `>` operator to be defined, and a set of classes written by someone else that didn't include any overloaded operators but did include a function for comparison, you might specialize your template to handle these classes separately.

Template Partial Specialization

Partial template specialization stems from similar motives as full specialization as described above. This time, however, instead of implementing a class for one specific type, you end up implementing a template that still allows some parameterization. That is, you write a template that specializes on one feature but still lets the class user choose other features as part of the template. Let's make this more concrete with an example.

Going back to the idea of extending the concept of vectors so that we can have a sortedVector, let's think about how this might look: we'll need a way of making comparisons. Fine; we can just use `>` if it's been implemented, or specialize if it hasn't. But now let's say that we wanted to have pointers to objects in our sorted vector. We could sort them by the value of the pointers, just doing a standard `>` comparison (we'll have a vector sorted from low to high):

```
template <typename T>
class sortedVector
{
    public:
        void insert (T val)
        {
            if ( length == vec_size )    // length is the number of elements
            {
                vec_size *= 2;    // we'll just ignore overflow possibility!
                vec_data = new T[vec_size];
            }
            ++length;    // we are about to add an element
        }
};
```

```

        // we'll start at the end, sliding elements back until we find the
        // place to insert the new element
        int pos;
        for( pos = length; pos > 0 && val > vec_data[pos - 1]; --pos )
        {
            vec_data[pos] = vec_data[pos - 1];
        }
        vec_data[pos] = val;
    }
    // other functions...
private:
    T *vec_data;
    int length;
    int size;
};

```

Now, notice that in the above for loop, we're making a direct comparison between elements of type T. That's OK for most things, but it would probably make more sense to have sorted on the actual object type instead of the pointer address. To do that, we'd need to write code that had this line:

```
for( pos = length; pos > 0 && *val > *vec_data[pos - 1]; --pos )
```

Of course, that would break for any non-pointer type. What we want to do here is use a partial specialization based on whether the type is a pointer or a non-pointer (you could get fancy and have multiple levels of pointers, but we'll stay simple).

To declare a partially specialized template that handles any pointer types, we'd add this class declaration:

```

template <typename T>
class sortedVector<T*>
{
public:
    // same functions as before. Now the insert function looks like this:
    insert( T *val )
    {
        if ( length == vec_size )    // length is the number of elements
        {
            vec_size *= 2;    // we'll just ignore overflow possibility!
            vec_data = new T[vec_size];
        }
        ++length;    // we are about to add an element

        // we'll start at the end, sliding elements back until we find the
        // place to insert the new element
        int pos;
        for( pos = length; pos > 0 && *val > *vec_data[pos - 1]; --pos )
        {
            vec_data[pos] = vec_data[pos - 1];
        }
        vec_data[pos] = val;
    }

private:
    T** vec_data;
    int length;
    int size;
};

```

There are a couple of syntax points to notice here. First, our template parameter list still names T as the parameter, but the declaration now has a T* after the name of the class; this tells the compiler to match a pointer of any type with this template instead of the more general template. The second thing to note is that T is

now the type pointed to; it is **not** itself a pointer. For instance, when you declare a `sortedVector<int*>`, `T` will refer to the `int` type! This makes some sense if you think of it as a form of pattern matching where `T` matches the type if that type is followed by an asterisk. This does mean that you have to be a tad bit more careful in your implementation: note that `vec_data` is a `T**` because we need a dynamically sized array made up of pointers.

You might wonder if you really want your `sortedVector` type to work like this--after all, if you're putting them in an array of pointers, you'd expect them to be sorted by pointer type. But there's a practical reason for doing this: when you allocate memory for an array of objects, the default constructor must be called to construct each object. If no default constructor exists (for instance, if every object needs some data to be created), you're stuck needing a list of pointers to objects, but you probably want them to be sorted the same way the actual objects themselves would be!

Note, by the way, that you can also partially specialize on template arguments--for instance, if you had a `fixedVector` type that allowed the user of the class to specify both a type to store and the length of the vector (possibly to avoid the cost of dynamic memory allocations), it might look something like this:

```
template <typename T, unsigned length>
class fixedVector { ... };
```

Then you could partially specialize for booleans with the following syntax

```
template <unsigned length>
class fixedVector<bool, length> {...}
```

Note that since `T` is no longer a template parameter, it's left out of the template parameter list, leaving only `length`. Also note that `length` now shows up as part of `fixedVector`'s name (unlike when you have a generic template declaration, where you specify nothing after the name). (By the way, don't be surprised to see a template parameter that's a non-type: it's perfectly valid, and sometimes useful, to have template arguments that are integer types such as `unsigned`.)

A final implementation detail comes up with partial specializations: how does the compiler pick which specialization to use if there are a combination of completely generic types, some partial specializations, and maybe even some full specializations? The general rule of thumb is that the compiler will pick the most specific template specialization--the most specific template specialization is the one whose template arguments would be accepted by the other template declarations, but which would not accept all possible arguments that other templates with the same name would accept.

For instance, if you decided that you wanted a `sortedVector<int*>` that sorted by memory location, you could create a full specialization of `sortedVector` and if you declared a `sortedVector<int*>`, then the compiler would pick that implementation over the less-specific partial specialization for pointers. It's the most specialized since only an `int*` matches the full specialization, not any other pointer type such as a `double*`, whereas `int*` certainly could be a parameter to either of the other templates.